

должность · полный доступ

Администратор

У вас открыты все разделы системы и все действия. Кроме повседневной работы это означает ответственность за три вещи: **доступы сотрудников, справочники и объекты** и **кассу с заявками**. Эта инструкция ведёт по всем вашим сценариям — от выдачи доступа новому сотруднику до правки матрицы прав.

Все разделы и кнопки

Выдача доступов

Смета и касса

Матрица прав (UI + БД)

1 Что вам доступно

РАЗДЕЛ	ВАШИ ДЕЙСТВИЯ
 Рабочий экран	Сводные цифры, все задачи, создание и отмена задач, назначение исполнителей, удаление задач.
 Объекты	Создание и удаление объектов, карточка объекта, план-факт, доп. расходы, файлы и смета, график работ.
 Сотрудники	Добавление, карточка, блокировка и восстановление, привязка и отвязка аккаунта, удаление, постановка задачи из карточки.
 Снабжение	Создание заявок на материалы и обработка любых заявок: взять в работу, загрузить счёт поставщика, доставить на объект, архивировать, удалить.
 Финансы	Обработка финансовых заявок: одобрить, отклонить, выдать деньги; удаление заявок.
 Реестр	Сводная таблица расходов с фильтрами и выгрузкой в Excel.
 Диагностика	Журнал ошибок сотрудников: что падает, что повторяется чаще всего и у кого именно. Раздел только для чтения.
Кабинет	Свой баланс, авансовый отчёт (расход, возврат, Excel), мои заявки, мои задачи.

Диагностика: что падает у сотрудников

Раздел « **Диагностика**» — это журнал ошибок приложения. Приложение само записывает в базу необработанные исключения, отклонённые промисы и отказы сохранения из браузеров сотрудников, а раздел показывает их в удобном виде. До него о проблемах узнавали со слов («у меня не сохраняется»), хотя причина обычно лежала в консоли браузера — туда сотрудник не смотрит и скриншот оттуда не присылает.

В разделе есть фильтры **период** (сутки / неделя / месяц / 90 дней) и **вид события**, сводка (сколько записей, разных ошибок и у скольких сотрудников), срез « **Что повторяется чаще всего**», « **Последние записи**» и « **У кого падает**». Нажатие на строку открывает подробности записи: сотрудник, роль, версия приложения и ревизия оболочки, страница, браузер, текст ошибки, стек вызовов

и технический контекст. Именно по ревизии оболочки видно, работает ли сотрудник на свежей версии приложения.

Что важно знать. Раздел **только для чтения**: он ничего не меняет ни в заявках, ни в деньгах. Открыт **Администратору и Директору** — это внутренняя диагностика, а не рабочие данные. В журнале нет сумм, файлов и паролей: только текст ошибки, стек, адрес страницы и сведения о браузере. Записи старше **90 дней** удаляются сами. Если срезы пусты — либо ошибок не было, либо таблица журнала ещё не создана: раздел сам напишет об этом над срезами и назовёт файл `database/migrate-v2.9-ops-monitoring.sql` (раздел «9. Обновление базы данных»). Ограничение раздела: он показывает **последние 300 записей** периода и честно предупреждает, если записей больше — тогда сузьте период.

2 Выдача доступа новому сотруднику

- 1 Сотрудник сам открывает адрес приложения → вкладка «**Регистрация**» → указывает email и пароль (минимум 6 символов). До привязки он видит только экран «Доступ не активирован».
- 2 Вы открываете «**👤 Сотрудники**» → «**+ Добавить сотрудника**» и заполняете: **ФИО, должность** (см. ниже), **телефон**, при необходимости — заметки. Сохранить.
- 3 В карточке сотрудника нажмите «**🔗 Привязка аккаунта**» и укажите email, под которым он зарегистрировался.
- 4 Попросите сотрудника нажать «**🔄 Обновить**» на экране «Доступ не активирован» (или войти заново) — приложение откроется на его рабочем экране.

Должность — это роль. Список из семи значений: Директор, Администратор, Главный инженер, Прораб, Снабженец, Инженер ПТО, Финансист. От неё зависят и разделы, и кнопки. Если указать должность вне списка, интерфейс останется «минимальным».

Должность «Финансист». Такой сотрудник получает «**💼 Рабочий стол**»: он выдаёт деньги по заявкам, одобренным директором (сумма списывается с его подотчёта), и отмечает оплату счетов поставщиков («**📄 Счета на материалы**»). Заявки он не создаёт и не согласует. Если финансиста в штате нет, выдача по заявкам остаётся на директоре, а счета снабжения оплачивать некому — должность стоит завести и привязать к аккаунту.

Первый запуск системы: пока в базе нет ни одного привязанного Администратора, первый вошедший пользователь получает доступ автоматически — так настраивается система «с нуля». Сразу после этого привяжите себя к карточке Администратора и создайте остальных.

Смена должности

Должность меняется в карточке сотрудника. Если сотрудник переходит на другую роль, после сохранения попросите его перезайти (или обновить страницу) — набор разделов пересчитается под новую должность.

3 Карточка сотрудника: блокировка, восстановление, удаление

В карточке видно ФИО, должность, телефон, заметки, баланс подотчёта, его объекты (где он прораб) и актуальные задачи. Оттуда же ставятся задачи и выполняются действия с аккаунтом.

ДЕЙСТВИЕ	ЧТО ПРОИСХОДИТ
 Заблокировать	Доступ закрывается сразу, данные (заявки, расходы, задачи) сохраняются. В форме указывается причина, например «Отпуск» или «Увольнение». Это правильный способ «отключить» сотрудника.
Восстановить	Возвращает статус «активный», но требует повторной привязки аккаунта — привяжите email заново, иначе сотрудник снова увидит «Доступ не активирован».
Отвязать аккаунт	Карточка сотрудника остаётся, но вход под прежним email перестаёт открывать данные. Используется при смене личного email.
 Удалить	Необратимо. Данные, привязанные к сотруднику (расходы, авторство заявок), могут стать «ничьими». Для уволившихся используйте блокировку, а не удаление.

Осторожно: перед удалением сотрудника убедитесь, что у него нет незакрытого подотчёта (баланс должен быть 0,00) и незавершённых заявок.

4 Касса, подотчёт и финансовые заявки

Обработка заявки на подотчёт

- 1 Раздел « **Финансы**» → фильтр « **Ожидают**».
 - 2 Откройте заявку: видны объект, раздел, позиции работ, итог и комментарий.
 - 3 «**Одобрить**» — согласовано (статус ) , деньги ещё не выданы; либо «**Отклонить**» — статус  и причина в комментарии.
 - 4 После передачи наличных нажмите «**Выдать**» (статус ). С этого момента сумма появляется в балансе сотрудника и её нужно отчитаться расходами или возвратом.
- **Выдача подотчёта** доступна и вне заявки — из карточки сотрудника, если деньги выданы без предварительной заявки.
 - **Балансы всех сотрудников** видны в разделе « Сотрудники» и в кабинете:  отрицательный — долг сотрудника,  положительный — у него остаток на руках.
 - **Свои расходы** вносятся так же, как у всех: «Кабинет» → « Авансовый отчёт» → « Внести расход» / « Вернуть».

Что важно в учёте

- Одобрение ≠ выдача: не отмечайте деньги выданными, пока они реально не на руках — иначе баланс сотрудника «похудеет» раньше времени.
- Перед выдачей проверьте, нет ли у сотрудника старого непогашенного долга.
- Заявку, созданную по ошибке, правильнее **отклонить**, а не удалять: история остаётся прозрачной.

5 Объекты и смета

Создание объекта

- 1 « **Объекты**» → « **Добавить объект**».

- 2 Укажите **название** и **ответственного прораба** — от этого зависит, какой сотрудник будет видеть объект как «свой» и сможет закрывать его разделы.

Удаление объекта убирает вместе с ним все разделы, включая служебный «Доп. расходы», а значит — привязку расходов. Используйте только для объектов, созданных по ошибке.

Загрузка и обновление сметы (Excel)

- 1 Карточка объекта → подвкладка « **Файлы**» → блок « **Смета объекта**» (видят Администратор, Главный инженер и Инженер ПТО).
- 2 Нажмите загрузку и выберите файл **.xlsx**. Система сама создаст **разделы** и плановые суммы работ и материалов.
- 3 Проверьте подвкладку « **План-факт**»: разделы и план должны совпасть со сметой.
- 4 Скачать файл обратно можно кнопкой просмотра сметы, отчёт — выгрузкой в PDF.

Повторная загрузка сметы — это обновление, а не создание заново: строки сопоставляются по названию раздела, обновляются только плановые суммы. Даты, уже проставленные в графике, сохраняются. Разделы, которых в новом файле нет, удаляются — поэтому перед обновлением уберите лишние строки и проверьте названия.

Служебный раздел «Доп. расходы» система создаёт сама в каждом объекте — в Excel-файле его быть не должно. Если такое название случайно встретится в смете, строка будет пропущена с предупреждением.

Удаление сметы («») доступно только вам: остальные роли файл сметы видят, но удалить не могут. Служебный раздел «Доп. расходы» при удалении сметы сохраняется, чтобы не потерять расходы вне сметы.

6 **Снабжение, реестр и график**

Заявки на материалы

- Создать заявку можно из шапки (« **Заказ материалов**») или внутри раздела (« **Создать заявку**»): объект → раздел → позиции (наименование, единица, количество) → сохранить. Номер  присваивается автоматически.
- Вы — один из двух, кто **обрабатывает** заявки (второй — снабженец): «Взять в работу», « **Счёт от поставщика**» (файл, поставщик, цены), затем « **Доставлено на объект**» с фактическими ценами по позициям и способом оплаты (« **фирма** или  **из подотчёта**).
- **Архив** — способ убрать заявку из активных без потери истории; суммы остаются в реестре.
- **Если сметы нет** и нужно оформить незапланированную покупку — в разделе выбирайте « **Доп. расходы**» (группа «Вне сметы»).

Реестр расходов

- Фильтры: объект, раздел, категория (материалы / работы / доставка / прочее), оплата (оплачено / в долг), сотрудник, период «Дата с — Дата по».
- Заголовок показывает « **Записей**» и « **Итого**» по текущему фильтру.

- « Excel» — выгрузка того, что видно после фильтров: готовая таблица с датами и суммами, удобно отправлять в бухгалтерию.

График работ

- Подкладка « График» карточки объекта: полосы разделов. Плановая полоса закрашена цветом выбранной схемы (по умолчанию фирменный красный), фактические — синие (в срок) и красные (с опозданием).
- Даты плана правятся в окне « Редактирование дат разделов» или перетаскиванием полос.
- Клик по полосе открывает детали раздела; закрытие раздела (фактическая дата) делает прораб по своему объекту, при необходимости скорректируйте дату.
- Диаграмма выгружается в PDF — удобно приложить к отчёту или распечатать для планёрки.

7 Задачи и сотрудники

- **Создать задачу:** « Рабочий экран» → « Создать задачу»: заголовок, исполнитель (Прораб, Снабженец, Инженер ПТО), описание, объект и раздел, приоритет, дедлайн.
- **Из карточки сотрудника:** кнопка « Поставить задачу» — исполнитель подставляется автоматически. Кнопка есть только у роли, которой разрешена постановка задач «от сотрудника».
- **Контроль:** фильтры « Активные», « Новые», « В работе», « Выполненные», « Все». Сводка показывает в работе, просроченные (дедлайн нарушен) и выполненные за 30 дней — цифры кликабельны и открывают список задач.
- **Действия:** взять в работу, выполнить с фотоотчётом, отменить (у вас — любую задачу), удалить, добавить комментарий.
- **Фотоотчёт** — обязательная часть закрытия задачи: по нему видно, что работа сделана, а не «сказано, что сделана».

8 Администрирование прав: два уровня

Права в системе заданы в двух независимых местах. Это важно понимать, если вы «открываете доступ» роли:

УРОВЕНЬ	ГДЕ ЗАДАЁТСЯ	ЧТО ДЕЛАЕТ
Интерфейс	Файл <code>js/permissions.js</code> : матрица прав по ролям, требования к вкладкам и урезанный интерфейс отдельных ролей.	Показывает или скрывает разделы и кнопки. Удобство, а не защита.
База данных	Политики доступа (RLS) в Supabase на таблицы и на хранилища файлов.	Настоящая защита: даже при обходе интерфейса данные недоступны.

Правило: меняете права роли — правьте **в обоих местах**. Иначе сотрудник либо увидит кнопку, которая не сработает, либо получит доступ к данным, которые вы хотели закрыть.

Проверка после изменения прав: попросите тестового сотрудника этой роли перезайти в приложение и убедитесь, что доступны ровно те разделы и кнопки, которые должны быть. Комментарии-пояснения ко всем решениям по правам ведутся в `README.md` репозитория — держите их в актуальном состоянии.

Список должностей для форм совпадает с ключами матрицы прав. Добавляя новую должность, нужно добавить её и в справочник, и в матрицу, иначе роль получит доступ «по минимуму».

9 Обновление базы данных (миграции)

Приложение и база обновляются отдельно. Код приложения — это статика, а колонки в базе добавляются **один раз вручную** в Supabase. Если обновить только код, новые функции сломаются: приложение будет сохранять счёт, доставку и оплату заявки в колонки, которых в базе ещё нет.

ЧТО ВИДИТ СОТРУДНИК	ЧТО ЭТО ЗНАЧИТ
Тост «Не удалось сохранить счёт: База данных не обновлена: в таблице «orders» нет колонки «payment_status». Примените database/migrate-v2.4.sql...»	Не применена миграция v2.4.0 (или применилась наполовину): в таблице <code>orders</code> нет нужных колонок. Приложение само переводит техническую ошибку базы на русский.
«Ошибка закрытия заявки: ... Could not find the 'delivered_at' column of 'orders' in the schema cache»	То же самое: нет колонки с датой доставки.
«Ошибка закрытия заявки: База отклонила запись: в таблице «orders» сработало ограничение «orders_status_check» — в списке статусов нет «Доставлено на объект»...»	Колонки уже есть, но на <code>orders.status</code> осталось старое CHECK-ограничение со списком статусов без нового статуса <code>delivered</code> . Лечится повторным запуском того же файла миграции (блок 4) — см. предупреждение ниже.
Тост «Не удалось сохранить счёт: База данных не обновлена: в таблице «orders» нет колонки «invoice_price_mode» (или «vat_rate», «own_delivery_charge»). Примените database/migrate-v2.5.sql...»	Не применена миграция v2.5.0 : в базе нет колонок НДС и своей доставки. Приложение называет именно нужный файл — запуск v2.4.0 здесь не поможет, ошибка повторится.
Тост « База отклонила запись: в таблице «cash_requests» сработало ограничение «cash_requests_status_check» — в списке статусов нет нужного значения («На доработке» или «В архиве»). Примените database/migrate-v2.6.sql...» (директор нажал «  На доработку»), а у автора заявки — «Не удалось убрать заявку в архив: ...» при нажатии «  В архив»	На <code>cash_requests.status</code> осталось старое CHECK-ограничение: в списке нет <code>revision</code> («  На доработку») и <code>archived</code> («  В архив»). Колонки при этом на месте, поэтому самопроверки v2.4.0 и v2.5.0 показывают <code>ok</code> и ничего не подозревают. Лечится файлом <code>database/migrate-v2.6.sql</code> — см. подраздел ниже.
Жёлтая плашка в «  Снабжении»: «Заявки не загрузились...»	Список заявок не загрузился. Плашка показывает причину, которую вернула база, — пустого списка без объяснений больше нет.
Жёлтая плашка в «  Реестре»: «Заявки на материалы не загрузились...»	База отвечает на запрос реестра ошибкой (чаще всего — про отсутствующую колонку).
Красная строка в блоке «  Счета на материалы» на рабочем столе финансиста: «Не удалось загрузить счета...»	Очередь счетов не загрузилась: счета не пропали, но база не отдала данные.

1 Supabase → **SQL Editor** → **New query**. Перед запуском сделайте бэкап (Database → Backups).

- 2 Откройте в репозитории файл `database/migrate-v2.4.sql`, скопируйте его **целиком** в редактор и нажмите **Run** — один раз, а не по блокам.
- 3 Посмотрите результаты запросов и вкладку **Notices**. Должно быть два результата: самопроверка колонок — **8 строк со статусом ok**, и проверка статусов — строка `orders_status_check = ok`. Если есть `MISSING` — в «Notices» будет точная ошибка базы по этой колонке.
- 4 Попросите сотрудников обновить приложение (Ctrl+F5) и повторить действие.

Миграция v2.5.0: НДС и своя доставка

В версии 2.5.0 у счёта появились **НДС (ПДВ)** и учёт **своей доставки**, поэтому нужен свой файл — `database/migrate-v2.5.sql`. Без него снабженец не сохранит счёт: база отклонит запись из-за колонок НДС.

- 1 Supabase → **SQL Editor** → **New query**: скопируйте `database/migrate-v2.5.sql` **целиком** → **Run**.
- 2 Проверьте вкладку **Notices** и результат запроса: в конце файла идёт самопроверка — **12 строк со статусом ok** (6 колонок в `orders`, 4 в `order_items`, 2 в `cash_operations`). Где `MISSING` — запустите файл ещё раз целиком и смотрите Notices: там точная ошибка базы по колонке.
- 3 В Notices будет ещё два сообщения о **старых строках доставки**: сколько строк «Доставка» помечено как доставка поставщика, а «Доставка компании» — как своя. Суммы от этого не меняются, меняется только признак (по нему код отличает доставку поставщика от своей).
- 4 Попросите сотрудников обновить приложение (Ctrl+F5): в окне счёта появятся блоки « ПДВ в счёте» и « Чья доставка», а в « Реестре» — колонка «в т.ч. ПДВ».

Порядок применения: если миграция v2.4.0 ещё не применена — сначала `database/migrate-v2.4.sql` (счёт поставщика и статусы), затем `database/migrate-v2.5.sql` (НДС и своя доставка), затем `database/migrate-v2.6.sql` (архив заявок на финансы). Файлы независимы, и повторный запуск любого из них безвреден — см. примечания ниже.

Приложение называет нужный файл само. Если база не знает колонку v2.5.0, в подсказке будет именно `database/migrate-v2.5.sql`, а не v2.4.0: раньше на любую отсутствующую колонку приложение отвечало «примените migrate-v2.4.sql», и администратор запускал не тот файл, получая ту же ошибку второй раз.

Повторный запуск безопасен: команды миграции идут с `if not exists`. Файл `database/schema.sql` — это схема для **чистой** установки; на работающей базе применяют только миграции.

«Наполовину применённая» миграция — почему она случается: SQL Editor останавливает скрипт на **первой** ошибке, поэтому восемь колонок могли добавиться частично (на боевой базе так и вышло: 4 из 8). Раздел «Снабжение» после этого работает странно — счёт не сохраняется, у финансиста пусто. Поэтому в файле каждая колонка добавляется отдельным защищённым блоком, а в конце идёт самопроверка. Если самопроверка показала `MISSING` — просто запустите файл ещё раз целиком.

«Заявка не закрывается»: старое ограничение статусов. У таблицы `orders` с прежних версий есть CHECK-ограничение на колонку `status` — список допустимых значений. В версии 2.4.0 у заявки появился новый статус `delivered` («🚚 Доставлено на объект»), и если ограничение не обновить, база отклоняет закрытие закупки с ошибкой `new row for relation "orders" violates check constraint "orders_status_check"`. В миграции это делает **блок 4**: он снимает старое ограничение и ставит новое со списком `new | in_progress | delivered | closed | archived`. Если сотрудники жалуются, что заявка не закрывается, а самопроверка колонок — «8 ok»: запустите `database/migrate-v2.4.sql` целиком ещё раз (или, быстрее, служебный файл `database/fix-orders-status-check.sql`) и проверьте, что результат показывает `orders_status_check = ok`.

Важно на будущее: если в новой версии появится ещё один статус заявки, его нужно добавить и в это ограничение (миграцией), иначе база отклонит запись. Список статусов в коде — `js/config.js`, ключ `CONFIG.ORDER_STATUS`.

Проверка после обновления: создайте тестовую заявку на материалы, возьмите её в работу, загрузите счёт, нажмёте «🚚 Доставлено на объект» и убедитесь, что заявка появилась в «📁 Реестре материалов» со статусом «⌚ Ожидает оплаты», а на рабочем столе финансиста — в блоке «📄 Счета на материалы».

Миграция v2.6.0: архив заявок на финансы

В версии 2.6.0 у заявок на **финансы** появились те же привычные действия, что и у заявок на материалы: автор убирает **законченную** заявку со своего рабочего экрана кнопкой «📁 В архив» (только свою и только выданную или отклонённую), а директор возвращает заявку автору кнопкой «✎ На доработку». **Колонок** для этого не нужно: это новые значения в существующей колонке `cash_requests.status`, поэтому файл миграции не добавляет ни одной колонки. Нужен он ровно из-за одного: на боевой базе со старых версий на этой колонке стоит CHECK-ограничение, в списке которого значений `archived` и `revision` нет. Ограничение переживает обновление приложения: пока его не обновить, кнопки нажимаются, а база молча отклоняет запись.

- 1 Supabase → **SQL Editor** → **New query**: скопируйте `database/migrate-v2.6.sql` целиком → **Run**.
- 2 В конце файла идёт самопроверка: нужны **шесть строк со статусом ok** — по одной на каждый статус (`pending`, `approved`, `revision`, `rejected`, `issued`, `archived`). Где `MISSING` — запустите файл ещё раз целиком и смотрите **Notices**: там точная ошибка базы (чаще всего нет прав на `alter table`).
- 3 Первый результат запроса показывает, какое ограничение стоит сейчас (имя и список значений) — по нему видно, что было до и после. Если в **Notices** есть строка `НЕ СНЯТО` — ограничение снять не удалось, пришлите эту строку разработчику.
- 4 Попросите сотрудников обновить приложение (Ctrl+F5) и повторить действие. Файл в конце просит базу перечитать схему (`notify pgrst, 'reload schema'`), иначе приложение ещё несколько минут считает, что статуса в базе нет.

Самопроверки v2.4.0 и v2.5.0 здесь не помогают. Они проверяют **колонки**, а v2.6.0 ни одной колонки не добавляет: в отчётах будет «8 ok» и «12 ok», хотя база по-прежнему отклоняет новый статус. Показатель — только `cash_requests_status_check` в самопроверке v2.6.0. Это же ограничение мешало и кнопке « На доработку»: статус `revision` появился ещё в v2.2.0, но старое ограничение не пропускает и его — поэтому файл разрешает сразу оба значения. Короткий файл `database/fix-cash-requests-status-check.sql` делает только половину работы (одно значение, `revision`) — для архива нужен **этот** файл.

«База отклонила запись» — это про ограничение, а не про колонку. Ошибка `new row for relation "cash_requests" violates check constraint "cash_requests_status_check"` (код 23514) означает ровно устаревшее ограничение: заявка никуда не «зависла», изменение просто не сохранилось — после миграции сотрудник повторяет действие. Повторный запуск `database/migrate-v2.6.sql` безопасен: файл снимает ограничение и ставит заново с теми же шестью значениями, а если `archived` уже разрешён — ничего не делает. **К заявкам на материалы это не относится:** `orders.status` знает `archived` с v2.4.0, архив прораба по закупкам работает без этого файла.

Миграции v2.7–v2.9: права в базе, команды, индексы, журнал ошибок

После v2.6.0 в базе появились ещё пять файлов. Порядок важен только у первых двух.

ФАЙЛ	ЧТО ДЕЛАЕТ
<code>database/migrate-v2.7-rls-finance.sql</code>	Права внутри базы для заявок на финансы и операций подотчёта: 15 политик (RLS), права для новых аккаунтов и функции, по которым база узнаёт сотрудника. Нужен до v2.8.0.
<code>database/migrate-v2.8-finance-rpc-audit.sql</code>	Команды вместо прямой записи: заявки на финансы создаются и обрабатываются четырьмя серверными командами (RPC), прямая вставка в <code>orders</code> и <code>cash_requests</code> закрыта, ведётся журнал <code>audit_log</code> с ключом идемпотентности — двойное нажатие «Одобрить» не создаёт вторую операцию. Требуется v2.7.0.
<code>database/migrate-v2.9-scale-indexes.sql</code>	Скорость при больших объёмах: 28 индексов под страничные списки и фильтры, которые считает база. Колонок и данных не трогает — можно запускать в любой момент.
<code>database/migrate-v2.9-ops-monitoring.sql</code>	Журнал ошибок для раздела «  Диагностика»: таблица <code>app_errors</code> и команда записи <code>rsk_log_app_errors</code> . Пишет только вошедший сотрудник, читают Администратор и Директор, записи хранятся 90 дней. Требуется v2.7.0.
<code>database/migrate-v2.9-registry-view.sql</code>	« Реестр» целиком: вид <code>registry_rows</code> (строки таблицы) и команда <code>registry_totals</code> — итоги «Записей» и «Итого» считаются по всему набору, а не по видимой странице. Можно запускать в любой момент.

Как проверять: каждый файл заканчивается самопроверкой — в конце нужны строки со статусом `ok`. Если база отвечает ошибкой прав, запускайте файл под аккаунтом владельца проекта; а если раздела « Диагностика» нет, приложение само назовёт файл миграции журнала ошибок.

10 Чек-лист администратора

Ежедневно

- ✓ Проверить «💰 Финансы» → «🔴 Ожидают»: нет ли заявок, которые ждут решения.
- ✓ Проверить «📦 Снабжение» → «🔴 Новые»: заявки не должны «висеть» без обработки.
- ✓ Посмотреть отрицательные балансы подотчёта и напомнить сотрудникам об отчётах.
- ✓ Заглянуть в «🔗 Диагностика»: если у сотрудников сегодня были ошибки — уточнить, что они делали в этот момент (текст ошибки и стек приложение уже собрал).

При новом сотруднике

- ✓ Карточка создана, должность выбрана из списка, телефон указан.
- ✓ Аккаунт привязан, сотрудник зашёл и увидел свой рабочий экран.
- ✓ Выдан адрес приложения и инструкция по установке (папка «Документация»).

При увольнении

- ✓ Подотчёт закрыт: расходы внесены или остаток возвращён (баланс 0,00).
- ✓ Незавершённые заявки и задачи переданы другому сотруднику.
- ✓ Сотрудник **заблокирован** (не удалён) с причиной «Увольнение».

Ежемесячно

- ✓ Сверить реестр расходов с бухгалтерией (выгрузка в Excel).
- ✓ Проверить, что должности всех сотрудников совпадают с реальными ролями.
- ✓ Проверить даты графика: не остались ли разделы без плановых или фактических дат.
- ✓ Просмотреть «🔗 Диагностика» → «💬 Что повторяется чаще всего» за месяц: повторяющаяся ошибка — это баг, о котором стоит сообщить разработчику, а не «разовый сбой».

Помните: вы — единственный, кто может удалять записи и менять права. Все действия подписаны вашим аккаунтом, поэтому спорные ситуации разбираются по истории в системе, а не «по памяти».